SECTION 6 · SUBSECTION 6.1
DESIGN DECISIONS

6.1.1 INTRODUCTION.

This section separates what the code does from what the records say about why. Each entry carries a basis mark. **OBSERVED** means the behaviour is directly visible in the working tree. **DOCUMENTED** means a repository record states the rationale, usually the decision log. **INFERRED** means the consequence or motive is this handbook's reading and is not asserted by the project. The distinction matters because several remembered decisions no longer match the implementation (figure 6.1-1).

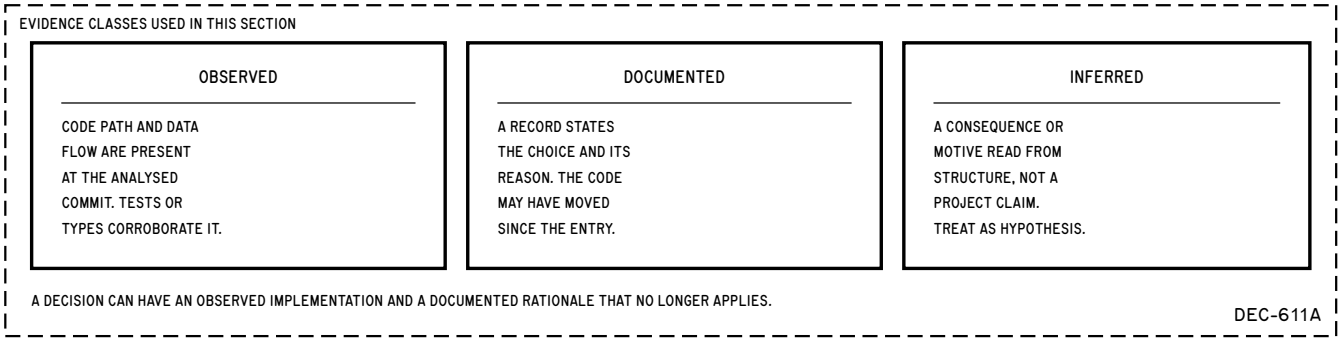


Figure 6.1-1. How Each Claim in This Section Is Grounded

DEC

6.1.2 ARCHITECTURAL DECISIONS.

The system is organised around one mutable state object and a fixed stage order. That choice keeps every intermediate value inspectable and makes the CLI and API share a single implementation, but it also couples all stages to one schema and makes partial re-runs coarse.

Table 6.1-1. Principal Decisions and Their Consequences

Decision	Basis	Consequence and cost
One shared state object	OBSERVED	Every stage is independently inspectable; schema changes ripple through all agents; no message contract or versioning
Agents do not import agents	OBSERVED	Stage replacement is local; orchestration knowledge concentrates in two runners that can drift
Parallel graph, complexity, retrieval	OBSERVED / DOCUMENTED	Wall-clock saving on I/O-bound retrieval; coupling must be deferred and reapplied after the join
Depth-first file cap	OBSERVED	Shallow entry points survive a sample; deep but central modules can be excluded without a warning in the document body
Hosted embeddings and chat	OBSERVED / DOCUMENTED	No local GPU or model download; the tool stops working without network, key and quota
Chroma collection per job	OBSERVED	Jobs cannot collide or read each other's code; collections accumulate because no retention job removes them
Redis optional, memory fallback	OBSERVED	One-command local start; memory mode cannot share state across processes and loses jobs on restart
Dual route families	OBSERVED / INFERRED	Older clients keep working; the two families return different codes and error shapes for the same condition
Markdown as primary output	OBSERVED / DOCUMENTED	Readable in any editor and usable as model context; structure is prose, so machine consumers must parse it or use JSON
Fail-open document sections	OBSERVED / INFERRED	A partial document is still delivered; a missing section can look like an empty finding rather than a failure
Client-side rendering and export	OBSERVED	No server download endpoint and no result storage beyond the job record; large payloads cross the network once
No URL router in the SPA	OBSERVED	Refresh returns to the landing screen; deep links and browser history are unavailable despite the installed router

6.1.3 Why the Pipeline Is Not a DAG Scheduler.

The three middle stages are independent after parsing, so a general scheduler was unnecessary. A fixed pool with explicit joins gives deterministic ordering of writes and avoids concurrent mutation of unrelated state fields. The cost is that a slow retrieval stage delays explainability even when the graph and scores are ready, because the join is a barrier. A finer-grained dependency graph could start explanation for files whose scores are final, but it would require per-file readiness tracking that the state object does not carry.

CAUTION

Per-job Chroma collections are removed only when *DELETE /jobs/{id}* is called, and that call is best-effort: a failure is logged and the record is deleted anyway. Nothing prunes on a timer and the client never calls it, so every analysis leaves an on-disk collection. Disk growth tracks runs, not repositories.

DECISION CHECK	REVIEWER
1 ASK - what does the code do	
2 ASK - what does the record claim	
√ MARK - observed, documented, inferred	
3 NAME - the cost, not only the benefit	